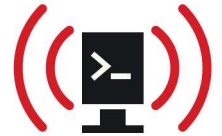




SYSMON

DEEP TELEMETRY



THE WINDOWS LOG WORTH READING

From Native Auditing to Attacker-Grade Evidence



INSTALL



CONFIGURE



COLLECT



HUNT



TUNE

WHAT YOU WILL MASTER

- ✓ What Sysmon records that the Security log never will
- ✓ The twenty-nine event types, and the nine that matter first
- ✓ Writing, testing and tuning a configuration file
- ✓ Parent-child chains, image hashes and DNS attribution
- ✓ Mapping Sysmon evidence onto an incident timeline
- ✓ Forwarding off the host before an attacker clears it
- ✓ Hands-on lab on a disposable Windows virtual machine

**REAL
COMMAND
LINES.
REAL
PARENTS.
REAL
HASHES.**

Event 4688 tells you a process started. Sysmon Event 1 tells you what it was told to do, who told it, and whether that binary has ever been seen before.



PROCESS

Event 1 with the full command line



NETWORK

Events 3 and 22, process to destination



REGISTRY

Events 12 to 14, persistence keys



FILES

Events 11 and 23, created and deleted



INTEGRITY

Events 8, 10 and 25, tampering

*THE SECURITY LOG TELLS YOU WHO LOGGED ON.
SYSMON TELLS YOU WHAT THEY RAN.*

DR. RAMYA SHAH

ASSISTANT PROFESSOR, SCHOOL OF CYBER SECURITY AND DIGITAL FORENSICS, NFSU

SYSMON FIELD HANDBOOK

*Log more.
Guess less.*

Incident Response Management
Unit 1 supplement • Sysmon v15.21
Issued 11 September 2026

System Monitor, almost always written **Sysmon**, is a free Windows service and kernel driver from the Sysinternals team at Microsoft. It does one job: it watches activity that Windows either does not record at all or records without the detail an investigator needs, and it writes that activity to its own event log. It is not antivirus. It blocks almost nothing, alerts on nothing, and will not stop an attacker. It produces evidence, which is a different thing and, in incident response, a more valuable one.

THE GAP IT FILLS

Windows native auditing was designed around accountability for logons and object access. That is why the Security log is excellent at telling you that `s.patel` authenticated from an external address at 02:19:41, and poor at telling you what that session then did. Event 4688 records that a process started, and on a default installation it does not record the command line at all, because command line auditing is a separate policy setting that most estates never enable.

Sysmon starts from the opposite premise. Its Event 1 carries the full command line, the parent process and the parent's command line, the working directory, the user, a logon GUID that ties the process back to the session that spawned it, and a cryptographic hash of the image on disk. An analyst reading one Sysmon Event 1 knows more about an execution than a page of native events will tell them.

HOW IT WORKS

Sysmon installs two components. A Windows service, running as a protected process, subscribes to kernel callbacks and to Event Tracing for Windows; and a filter driver, loaded early in boot, which is why Sysmon can record activity that happens before most security software has started. Events are filtered against a configuration file you supply, and the survivors are written to the Windows event log.

Filtering inside the driver is the part worth remembering. Sysmon can see far more than any estate wants to store, so the configuration is not an optional refinement afterwards. It is the product. An unconfigured Sysmon is a disk-filling machine. A well-configured one is the most useful sensor on a Windows host.

WHERE THE EVENTS LAND

Applications and Services Logs > Microsoft > Windows > Sysmon > Operational

In the Event Viewer exercises you worked inside the Security log, Sysmon is a separate channel with its own retention. Any serious custom view merges the two, because the story only reads correctly when a 4624 and a Sysmon Event 1 sit next to each other in one time order.

THE SAME ACTIVITY, SEEN BY BOTH

THE QUESTION YOU ARE ASKING	WINDOWS SECURITY LOG	SYSMON
What command was run?	Event 4688, and only if command line auditing was enabled by policy beforehand	Event 1, always, with the parent's command line as well
Is this binary known?	Nothing. No hash is recorded.	Event 1 carries MD5, SHA1, SHA256 or IMPHASH, plus the signature status
Which process opened that connection?	Event 5156 gives an application path, with no parent and no user context	Event 3 gives the process, the user, both addresses, both ports and the resolved name
What did the host resolve before it connected?	Nothing	Event 22: the DNS query, the answer, and the process that asked for it
Did something inject into another process?	Nothing	Events 8 and 10: remote thread creation, and process access with the granted rights

NATIVE AUDITING ANSWERS "WHO". SYSMON ANSWERS "WHAT, AND ON WHOSE BEHALF".

Neither replaces the other. Sysmon does not see failed logons at all, so the 612 failures in the FS01 case would still have come from the Security log. Run both, forward both, read them in one timeline.

INSTALLING IT

Sysmon ships as a single executable. Run it elevated. The configuration can be supplied at install time and swapped later without a reinstall and without a reboot.

```
:: install with a configuration
sysmon64.exe -accepteula -i sysmonconfig.xml

:: swap the configuration later
sysmon64.exe -c sysmonconfig.xml

:: show what is loaded right now
sysmon64.exe -c

:: print the event schema
sysmon64.exe -s

:: remove it
sysmon64.exe -u force
```

TWO WAYS TO OBTAIN IT

The classic route is the Sysinternals download, currently **version 15.21, released 17 June 2026**. Since February 2026 Microsoft has also shipped Sysmon as an optional feature inside Windows 11, first seen in Insider builds 26220.7752 and 26300.7733. It is off by default, is enabled under Settings, System, Optional features, and then completed with `sysmon -i`. Any standalone install has to be removed first. The built-in version does not replace the download; it removes the packaging problem for large estates.

BEFORE YOU RECOMMEND IT TO AN ESTATE

Sysmon runs with kernel privileges and sees everything, including whatever a user types at a command prompt. That is why it is useful and also why it is governed. Deployment belongs in a change window, with a tested configuration, a log size and disk plan, a staff notification, and a named owner for the noise it will produce on day one. A sensor nobody has agreed to read is a cost with no benefit attached.

ANATOMY OF ONE EVENT 1

This is the collection step from the FS01 case, as Sysmon would have recorded it. Every highlighted field is one the Security log does not give you.

```
EventID:          1 (Process Create)
UtcTime:          2026-08-14 02:24:01.118
ProcessGuid:      {a1f4c0e2-8c41-68cd-2f01-000000000900}
ProcessId:        8512
Image:            C:\Program Files\7-Zip\7z.exe
OriginalFileName: 7z.exe
CommandLine:      7z.exe a -p -mhe=on C:\Users\Public\sr.7z D:\Shares\StudentRecords\
CurrentDirectory: C:\Users\s.patel\
User:             NFSU\s.patel
LogonGuid:        {a1f4c0e2-7b15-68cd-c491-2a0000000000}
LogonId:          0x2A91C4
IntegrityLevel:   High
Hashes:           SHA256=6B5A1D...C9F2, IMPHASH=3D7A1E4C9B02F5A8
ParentProcessId:  6988
ParentImage:      C:\Windows\System32\cmd.exe
ParentCommandLine: "C:\Windows\system32\cmd.exe"
ParentUser:       NFSU\s.patel
```

CommandLine	The whole case in one line. An archiver told to bundle a Restricted share into a password-protected file in a world-readable folder. No native event on a default build carries this.
LogonGuid	Ties this process to the 02:19:41 remote logon. Every process the attacker spawns inherits it, so one pivot event turns into a complete session tree rather than a pile of unrelated rows.
Hashes	SHA256 answers "has anyone seen this file before". IMPHASH hashes the import table rather than the bytes, so a recompiled variant of the same malware still matches.
ParentImage and ParentCommandLine	Parentage is where most detections actually live. 7z.exe under explorer.exe is an employee. Under cmd.exe at 02:24, inside a session from a foreign address, it is collection.

Sysmon emits twenty-nine event types plus an error event. You do not need all of them. The **nine shaded rows** carry most of the detection value on an ordinary Windows estate and are where every new deployment should begin. Add the rest deliberately, one at a time, each with a reason written down.

ID	NAME	WHAT IT RECORDS	ID	NAME	WHAT IT RECORDS
1	ProcessCreate	Execution, full command line, parent, hashes	16	ConfigChange	Sysmon configuration was changed
2	FileCreateTime	Timestamp changed, a sign of timestamping	17	PipeEvent	Named pipe created
3	NetworkConnect	Connection, with the owning process	18	PipeEvent	Named pipe connected
4	ServiceStateChange	Sysmon itself started or stopped	19	WmiEvent	WMI event filter registered
5	ProcessTerminate	Process exit	20	WmiEvent	WMI consumer registered
6	DriverLoad	Driver loaded, with its signature	21	WmiEvent	Consumer bound to a filter
7	ImageLoad	DLL loaded into a process	22	DnsQuery	DNS query and answer, with the process
8	CreateRemoteThread	Thread created inside another process	23	FileDelete	File deleted, and archived to disk
9	RawAccessRead	Raw volume read, bypassing the file system	24	ClipboardChange	Clipboard capture
10	ProcessAccess	One process opened another, with granted rights	25	ProcessTampering	Image hollowing or herpaderping
11	FileCreate	File created or overwritten	26	FileDeleteDetected	Deletion logged without archiving
12	RegistryEvent	Key or value created or deleted	27	FileBlockExecutable	Executable write blocked
13	RegistryEvent	Value set, including Run keys	28	FileBlockShredding	File shredding blocked
14	RegistryEvent	Key or value renamed	29	FileExecutableDetected	Executable written to disk
15	FileCreateStreamHash	Alternate data stream created	255	Error	Sysmon failed at something

THREE THAT ARE NOT DETECTIONS

Events 4, 16 and 255 are about the sensor, not the host. A stream that goes quiet, a configuration that changed outside a change window, or a run of errors all mean your evidence has a hole in it. Alert on these first, because every other rule you write depends on them being silent.

TWENTY-SEVEN, TWENTY-EIGHT, TWENTY-NINE ARE DIFFERENT

FileBlockExecutable and FileBlockShredding are the only Sysmon events that stop something happening rather than record it. They are useful and they are also the only way Sysmon can break a production system. Pilot them on a small group and expect to tune.

READING THE CATALOGUE PROPERLY

Notice that three separate event IDs, 12, 13 and 14, all present themselves as RegistryEvent. Sysmon distinguishes them by the EventType field inside the record, so a detection rule written against the name alone will match more than you meant. The same applies to 17 and 18 for pipes, and to 19, 20 and 21 for WMI.

Notice also that 23 and 26 both describe a deletion. Event 23 copies the file into an archive directory before it disappears, which is extraordinary for forensics and dangerous for disk space. Event 26 records the same act and keeps nothing. Choose per path, never globally.

Event 25, ProcessTampering, deserves a place on every list. It fires when a running image no longer matches the file it was loaded from, which is what hollowing and herpaderping look like from the kernel.

Everything Sysmon is worth depends on one XML file. Two estates running the identical binary, one with a tuned configuration and one with the defaults, produce sensors of entirely different quality. This is the part students skip and practitioners spend years on.

INCLUDE OR EXCLUDE

Each event type gets a rule group that is either `include` or `exclude`, and the choice produces very different sensors. Exclude everything known-good and keep the rest: noisy, but it misses little. Include only what you decided matters: quiet, but blind to anything you did not anticipate. Mature estates run include lists for high-volume events such as image loads and registry writes, and exclude lists for process creation, where you want to see everything you have not explicitly forgiven.

```
<Sysmon schemaversion="4.82">
  <HashAlgorithms>SHA256,IMPHASH</HashAlgorithms>
  <EventFiltering>

    <!-- keep every execution except one
         very chatty signed binary -->
    <ProcessCreate onmatch="exclude">
      <Image condition="is">
        C:\Windows\System32\SearchIndexer.exe
      </Image>
    </ProcessCreate>

    <!-- only the connections worth storing -->
    <NetworkConnect onmatch="include">
      <Image condition="end with">powershell.exe</Image>
      <Image condition="end with">curl.exe</Image>
      <DestinationPort>4444</DestinationPort>
    </NetworkConnect>

  </EventFiltering>
</Sysmon>
```

Start from a published baseline, not an empty file. The community configurations from SwiftOnSecurity and from Olaf Hartong are the two the industry actually runs, and both are written to be read and argued with.

TUNING, IN THE ORDER THAT WORKS

- 1 Deploy to a handful of representative machines, not to the estate. One of each build image, one server, one developer workstation.
- 2 Run for a week and count events by ID and by image path. The top ten images will account for most of the volume and almost none of the value.
- 3 Exclude by the most specific field available. Exclude a full signed path, never a bare file name, or an attacker will simply name their binary to match your exclusion.
- 4 Measure again. A tuned workstation usually settles between two hundred and two thousand events per host per day.
- 5 Version the file, review it quarterly, and treat every change to it as a change to a security control.

HASHES ARE THE CHEAPEST WIN AVAILABLE

Set `HashAlgorithms` to `SHA256` and `IMPHASH`. `SHA256` lets you ask a threat intelligence service whether anyone in the world has seen this file. `IMPHASH` hashes the import table rather than the bytes, so two samples recompiled from the same source still match, which is how one Event 1 becomes a family of related detections.

ITS LIMITS, STATED PLAINLY

An attacker holding administrator rights can stop the service or unload the driver, and Events 4 and 16 are the records that say so. Sysmon does not decrypt traffic, does not read process memory, and holds no history from before the day you installed it. Like every local log it is only trustworthy once it has been forwarded somewhere the attacker does not control. That last sentence is the whole argument for Unit 3.

Disposable Windows virtual machine, snapshot taken before you begin, no network path to anything you care about. Nothing in this exercise is malicious, but every step of it looks malicious, which is the point: you are learning to read the evidence that ordinary behaviour and hostile behaviour both leave.

- 1 Install Sysmon with a published baseline configuration. Confirm the Operational channel exists and is filling.
- 2 Run five ordinary commands from a command prompt. Find each as an Event 1 and note the six fields you would put in a ticket.
- 3 From a Word document, trigger a macro that launches PowerShell. Compare the Sysmon Event 1 with the native 4688 for that same execution, field by field.
- 4 Run nslookup against three domains, then browse to one. Match each Event 22 to the Event 3 that follows it, using the process and the timestamp.
- 5 Write a Run key with `reg add`. Find the Event 13, and explain why your configuration either caught it or did not.
- 6 Stop the Sysmon service, find Event 4, restart it and find Event 16. Write two sentences on what an analyst should do when a host stops reporting.
- 7 Add one exclusion of your own, reload with `-c`, and prove with a before and after count that your rule did what you intended and nothing more.
- 8 Export the Operational channel to EVTX and open it on a second machine. That file is your evidence copy; note what makes it admissible and what does not.

SUBMISSION. One page: the six fields from step two, the comparison from step three, your exclusion rule, and your evidence that it was safe. Marks are for the justification, not for screenshots.

RULES FOR THE LAB

No live malware, no samples downloaded from anywhere, and no machine that touches the university network. Everything in this exercise is built from tools already present in Windows. If an exercise seems to require a real payload, you have misread it.

MARKING SCHEME

COMPONENT	WEIGHT
Correct field extraction, steps two and three	30
Event 22 to Event 3 correlation, step four	15
Exclusion rule, and why it is specific enough	30
Before and after evidence, step seven	15
Written clarity	10

REFERENCES

Microsoft, *Sysmon v15.21*, Sysinternals documentation, 17 June 2026. learn.microsoft.com/en-us/sysinternals/downloads/sysmon

Microsoft, native Sysmon optional feature in Windows 11, Insider builds 26220.7752 and 26300.7733, announced 4 February 2026.

NIST SP 800-61 Rev. 3, *Incident Response Recommendations and Considerations*, 2025.

NIST SP 800-92, *Guide to Computer Security Log Management*, on retention and forwarding.

SwiftOnSecurity, *sysmon-config*; O. Hartong, *sysmon-modular*. Community baseline configurations.

CERT-In Directions under section 70B(6), 28 April 2022, on retaining logs within Indian jurisdiction for 180 days.

WHERE THIS SITS IN THE SYLLABUS

Unit 1 establishes what an incident is and what evidence looks like. Sysmon is the instrument that produces that evidence on Windows. Collection, forwarding and correlation at estate scale belong to the SIEM material in Unit 3. This handbook stops at the host.